



La grande école du numérique pour tous

TrainingCyberBootcamp

Linux network services Project – Wazuh Implementation

Nadia BENBOUALI

MSc Cyber sécurité CYBERSUP

Année 2026-2027

Table des matières

1. Workstation Installation – Ubuntu Desktop	3
1.1 Configuration de la vm	3
1.2. Partitionnement du disque	3
2. Serveur-Infra Installation – Debian12.....	5
2.1 Configuration de la vm	5
2.3 Partie DHCP	6
2.5 Partie BIND9.....	8
2.6 Partie backup des fichiers de configuration :.....	9
2.4 Partie Wazuh	11
2.7.1 Partie démo Wazuh	13
2.7.2 Partie Intégration de YARA avec Wazuh	15

Introduction

Ce projet consiste en la conception d'une infrastructure Linux sécurisée intégrant des mécanismes avancés de détection et de réponse aux incidents.

Un serveur Debian 12 et une workstation Ubuntu ont été déployés dans un environnement virtualisé cloisonné, avec mise en place des services réseau essentiels (DHCP, DNS, GLPI) et supervision via Wazuh.

Des scénarios d'attaque (brute force SSH, dépôt de fichier malveillant) ont été simulés afin de tester la capacité du système à détecter, corrélérer et neutraliser automatiquement les menaces, notamment grâce à l'intégration de YARA et aux mécanismes d'Active Response.

1. Workstation Installation – Ubuntu Desktop

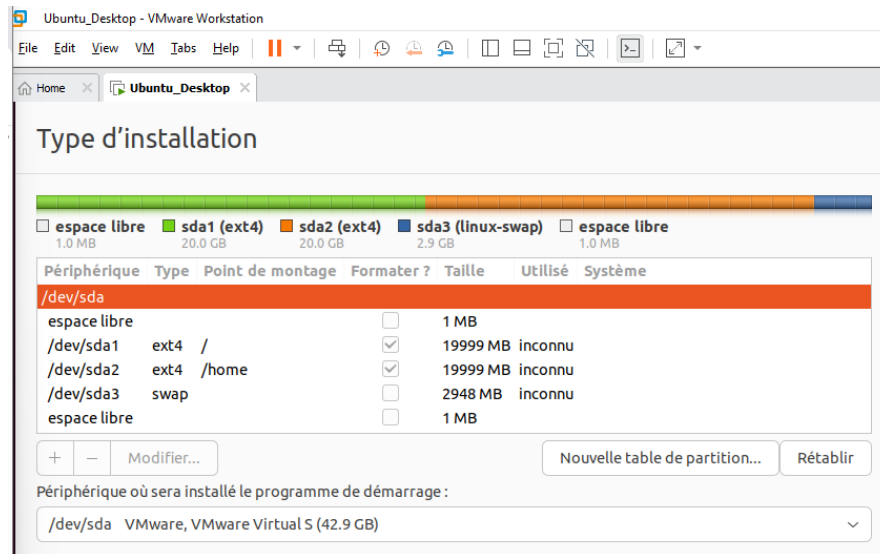
1.1 Configuration de la vm

- OS : Ubuntu Desktop 22.04 LTS
- RAM: 2 GB
- CPU: 2 cores
- Disk: 40 GB
- Network: Host-only (NAT au début)

1.2. Partitionnement du disque

Le disque a été partitionné comme suit et selon le cahier des charges :

- 512 MB – EFI System Partition
- 20 GB – Root partition (/) formatted in ext4
- 20 GB – Separate /home partition formatted in ext4
- 2 GB – Swap partition



Cette configuration permet de séparée le dossier /home,

Applications requises et installées :

- LibreOffice
- GIMP
- Web browser (Firefox)

Commande d'installation :

`sudo apt install libreoffice gimp -y`

Ce poste de travail est configuré pour être un client DHCP

une autre vm ubuntu server fera office de serveur DHCP

Pour ces installations te mise à jour la vm est en NAT, je la bascule en Host-only juste apres les installations

je fais un snapshot vmware à ce stade

```

attname enp2s1
nad@nad-UbunDesk:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:68:47:52 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.50.51/24 brd 192.168.50.255 scope global dynamic noprefixroute ens33
        valid_lft 595sec preferred_lft 595sec
    inet6 fe80::7600:28d0:9716:66f1/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

2. Serveur-Infra Installation – Debian12

2.1 Configuration de la vm

- OS :  `debian-12.5.0-amd64-netinst.iso` Attention Debian 13 non compatible avec la plateforme Wazuh
- RAM: 4 GB
- CPU: 2 cores
- Disk: 40 GB
- Network: 2 cartes réseau, NAT pour accès internet et Host-Only pour réseau et services internes

2.2 Partie GLPI

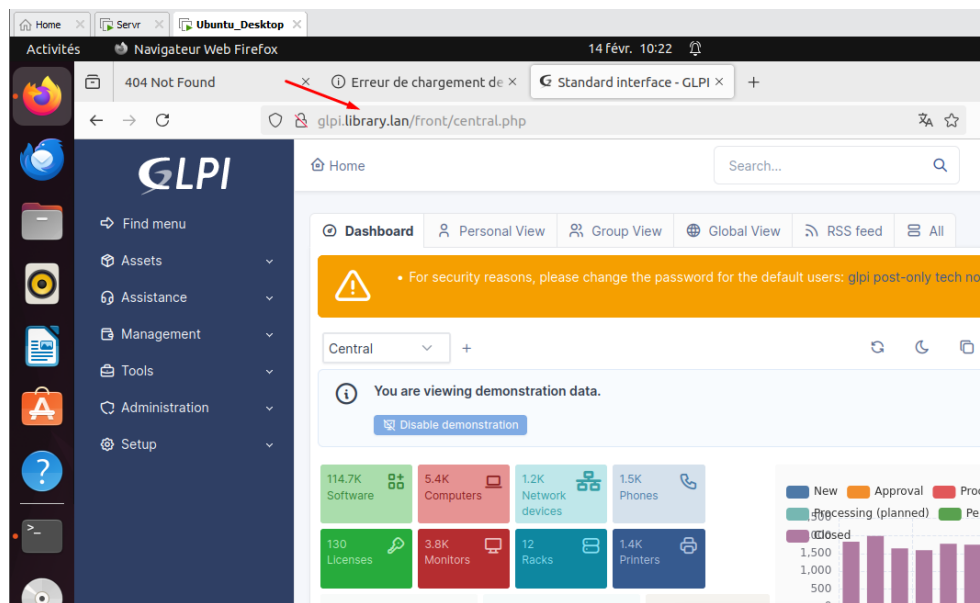
GLPI a été installé sur le serveur Debian (infra-server) et repose sur :

- Serveur web Apache2
- Base de données MariaDB
- PHP
- Hébergement dans `/var/www/glpi`

Le service est accessible via le DNS interne : `glpi.library.lan`

La résolution est assurée par le serveur DNS interne (BIND9) et distribuée aux clients via DHCP.

GLPI est uniquement accessible sur le réseau interne (interface Host-Only), garantissant son isolation vis-à-vis du réseau externe.



2.3 Partie DHCP

Faire écouter le dhcp sur la bonne carte réseau ens36 host only

```

nad@debian: ~
GNU nano 8.4 /etc/default/isc-dhcp-server *
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4_CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4_PID=/var/run/dhcpd.pid
#DHCPDv6_PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="ens36"
INTERFACESv6=""
  
```

Configuration du scope dans fichier /etc/dhcp/dhcpd.conf

```

subnet 192.168.50.0 netmask 255.255.255.0 {
    range 192.168.50.50 192.168.50.100;
    option routers 192.168.50.10;
    option domain-name-servers 192.168.50.10;
    option domain-name "library.lan";
}
  
```

État actuel de l'infrastructure

J'ai mis en place une architecture réseau interne basée sur deux interfaces réseau sur le serveur Debian.

Le serveur utilise :

- ens33 (NAT) : accès Internet temporaire pour l'installation des paquets et des mises à jour
- ens36 (Host-Only) : interface dédiée au réseau interne avec l'adresse IP statique 192.168.50.10/24

Tous les services internes (DHCP, DNS, GLPI) sont configurés pour fonctionner uniquement sur ens36, garantissant une isolation complète du réseau interne et évitant toute interférence avec le réseau physique (LAN).

Le serveur DHCP (isc-dhcp-server) distribue automatiquement des adresses IP dans la plage :

192.168.50.50 à 192.168.50.100

La workstation Ubuntu est configurée en adressage automatique (DHCP) et reçoit correctement une adresse IP interne (exemple : 192.168.50.51).

2.5 Partie BIND9

Bind9 installé sur infra-server (192.168.50.10)

Fonctionnement

- Résolution interne → gérée localement : Serveur DNS autoritaire pour library.lan
- Résolution externe → redirigée vers forwarders, (8.8.8.8, 1.1.1.1)
- DHCP distribue le DNS 192.168.50.10 aux clients

Zone configurée

Fichier :

/etc/bind/db.library.lan

Enregistrements :

ns.library.lan → 192.168.50.10

glpi.library.lan → 192.168.50.10

wazuh.library.lan → 192.168.50.20

Déclaration zone

Dans /etc/bind/named.conf.local

```
zone "library.lan" {  
    type master;  
    file "/etc/bind/db.library.lan";  
};
```

```
nad@nad-UbunDesk:~$ sudo nslookup glpi.library.lan  
Server:      127.0.0.53  
Address:     127.0.0.53#53  
  
Non-authoritative answer:  
Name:   glpi.library.lan  
Address: 192.168.50.10  
  
nad@nad-UbunDesk:~$ sudo nslookup wazuh.library.lan  
Server:      127.0.0.53  
Address:     127.0.0.53#53  
  
Non-authoritative answer:  
Name:   wazuh.library.lan  
Address: 192.168.50.20
```

La connexion passe désormais par la résolution DNS

Agent → wazuh.library.lan → DNS → 192.168.50.20

```
nad@nad-UbunDesk:~$ sudo grep address /var/ossec/etc/ossec.conf  
<address>wazuh.library.lan</address>  
nad@nad-UbunDesk:~$
```


2.6 Partie backup des fichiers de configuration :

Disque de sauvegarde dédié pour cela un second disque virtuel de 5 Go a été ajouté au serveur d'infrastructure.

Les étapes réalisées :

1. Création d'une partition (/dev/sdb1)
2. Formatage en ext4
3. Création d'un point de montage dédié :

/mnt/backup_disk

Le disque n'est pas monté en permanence.

Il est monté uniquement pendant l'exécution de la sauvegarde puis démonté automatiquement pour des raisons de sécurité.

- Script de sauvegarde

Un script Bash a été créé :

/usr/local/bin/backup_services.sh

- Contenu du script :

```
#!/bin/bash
```

```
DATE=$(date +%F)
```

```
BACKUP_DIR="/mnt/backup_disk"
```

```
BACKUP_FILE="$BACKUP_DIR/services_backup_$(date +%F).tar.gz"
```

```
# Monter le disque de sauvegarde
```

```
mount /dev/sdb1 $BACKUP_DIR
```

```
# Vérifier que le disque est monté
```

```
if mountpoint -q $BACKUP_DIR; then
```

```
tar -czf $BACKUP_FILE \
```

```
/etc/dhcp \
```

```
/etc/bind \
```

```
/etc/apache2 \
```

```
/etc/mysql\  
  
/var/www/glpi 2>/dev/null  
  
echo "Backup completed: $BACKUP_FILE"  
  
# Démonter le disque  
  
umount $BACKUP_DIR  
  
else  
  
echo "Backup disk not mounted. Backup aborted."  
  
fi
```

- Explication du fonctionnement

Le script réalise les actions suivantes :

1. Génère un nom de fichier incluant la date du jour.
2. Monte la partition de sauvegarde.
3. Crée une archive compressée (.tar.gz) contenant :
 - o La configuration DHCP
 - o La configuration DNS
 - o La configuration Apache
 - o La configuration MariaDB
 - o Le répertoire GLPI
4. Enregistre l'archive sur le disque secondaire.
5. Démonte automatiquement le disque une fois la sauvegarde terminée.

- Automatisation avec Cron

Pour rendre la sauvegarde automatique, une tâche cron a été configurée pour l'utilisateur root :

```
sudo crontab -e
```

Ligne ajoutée :

```
0 2 * * 0 /usr/local/bin/backup_services.sh
```

- Signification :

La sauvegarde s'exécute automatiquement chaque dimanche à 2h00.

2.4 Partie Wazuh

Une plateforme de sécurité informatique qui combine les fonctionnalités d'un SIEM et d'un XDR

```
14/02/2026 11:58:09 INFO: --- Summary ---
14/02/2026 11:58:09 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: lnm6A5vzGyI7rh1U*N1kgUje2Iiw4BlW
14/02/2026 11:58:09 INFO: --- Dependencies ----
14/02/2026 11:58:09 INFO: Removing gawk.
14/02/2026 11:58:12 INFO: Installation finished.
```

Je vais

Installer l'agent sur la workstation et le serveur -infra

Tester la communication agent → manager

- On ajout la workstation comme agent

renseigner l'ip du serveur Wazuh, donner un nom à l'agent, le tableau de bord génère une commande à exécuter dans le terminal la vm agent

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.5-1_amd64.deb
sudo WAZUH_MANAGER='192.168.1.167' WAZUH_AGENT_NAME='workstation_Ubuntu_Agent' dpkg -i ./wazuh-agent_4.7.5-1_amd64.deb
```

Agent Wazuh installé

```
wazuh-agent_4.7.5-1 100%[=====] 8,94M 5,08MB/s ds 1,8s
2026-02-14 12:31:39 (5,08 MB/s) - 'wazuh-agent_4.7.5-1_amd64.deb' enregistré [9378818/9378818]

[sudo] Mot de passe de nad :
Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 215920 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-agent_4.7.5-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.7.5-1) ...
Paramétrage de wazuh-agent (4.7.5-1) ...
nad@nad-UbunDesk:~$
```

Démarrer & vérifier

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable wazuh-agent
```

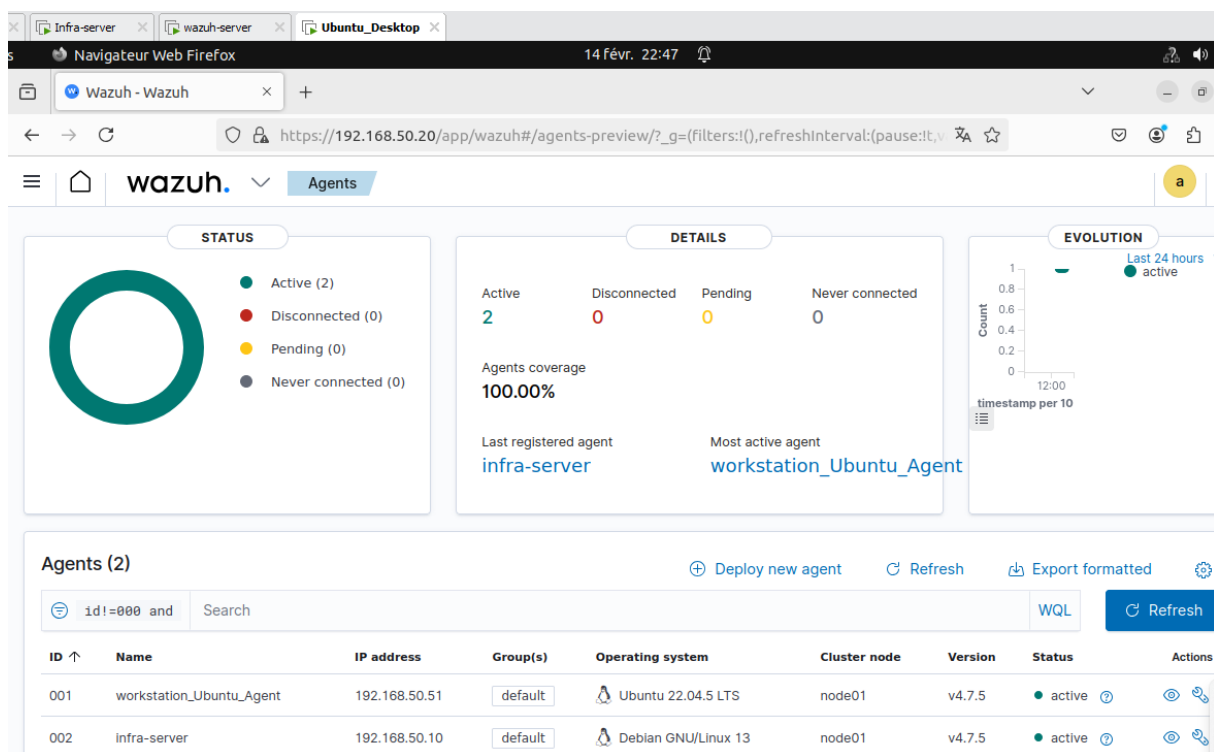
```
sudo systemctl start wazuh-agent
```

```

nad@nad-UbunDesk: ~
nad@nad-UbunDesk:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor p
   Active: active (running) since Sat 2026-02-14 12:37:37 CET; 21s ago
   Process: 6177 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (co
   Tasks: 5 (limit: 2204)
   Memory: 99.1M
   CPU: 12.434s
   CGroup: /system.slice/wazuh-agent.service
           └─6677 /bin/sh active-response/bin/restart.sh agent
             └─6681 /bin/sh /var/ossec/bin/wazuh-control restart
               └─6802 /var/ossec/bin/wazuh-execd
                 └─6804 sleep 1

```

sudo systemctl status wazuh-agent



2.7.1 Partie démo Wazuh

- Active Response simple et maîtrisée
- Déclencher sur tentatives SSH échouées
- Utiliser le script natif firewall-drop wazuh (légèrement customisé)
- Bloquer l'IP attaquante automatiquement (attaque depuis la workstation vers infra-server.)

`sudo nano /var/ossec/etc/ossec.conf`

Dans la section `<ossec_config>` :

```
<active-response>
  <command>firewall-drop-custom</command>
  <location>local</location>
  <rules_id>5760</rules_id>
  <timeout>300</timeout>
</active-response>
```

Dans la section `<command>` :

```
<command>
  <name>firewall-drop-custom</name>
  <executable>firewall-drop-custom.sh</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

5760= règle Wazuh pour échecs SSH multiples (il est important que ce numéro corresponde exactement à celui du `rules_id` dans l'événement security sur Wazuh sinon le firewall-drop ne déclenchera jamais)

timeout 300 = blocage 5 mn

Lorsqu'une tentative de brute force SSH est effectuée depuis la workstation vers le serveur infra, c'est l'agent Wazuh installé sur infra-server qui détecte les échecs d'authentification dans les logs. Cette alerte est envoyée au Wazuh manager, qui analyse l'événement et déclenche une *Active Response* (ici le script firewall-drop). Le manager envoie alors l'ordre au serveur infra d'ajouter une règle de pare-feu bloquant temporairement l'adresse IP source (celle de la workstation). Ainsi, ce n'est pas la workstation qui se bloque elle-même : c'est le serveur attaqué qui refuse désormais

Toute connexion provenant de l'IP identifiée comme suspecte.

```
nad@infra-server:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      all  --  192.168.50.51          0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
nad@infra-server:~$
```

Blocage des connexions depuis la 192.168.50.51 pendant 5 mn

```
nad@nad-UbunDesk:~$ ssh nad@192.168.50.10
ssh: connect to host 192.168.50.10 port 22: Connection timed out
nad@nad-UbunDesk:~$ ssh nad@192.168.50.10
ssh: connect to host 192.168.50.10 port 22: Connection timed out
nad@nad-UbunDesk:~$ ping 192.168.50.10
PING 192.168.50.10 (192.168.50.10) 56(84) bytes of data.
```

Résultat :

- Wazuh a détecté l'attaque brute force
- Active Response déclenchée, injecte une règle firewall

```
target     prot opt source                destination
DROP      all  --  192.168.50.51          0.0.0.0/0
```

- L'attaquant est réellement bloqué (pas réellement bloquée, c'est surtout les connexions vers serveur infra qui sont bloquées)

- Explications de la règle :

La règle DROP insérée dans la chaîne INPUT bloque tous les paquets provenant de l'adresse IP attaquante avant qu'ils n'atteignent le service SSH.

Le TCP handshake ne s'établit pas, ce qui provoque un timeout de connexion au lieu d'une erreur d'authentification.

Le script par défaut de Wazuh (firewall-drop) n'était pas entièrement compatible avec mon infra. J'ai donc implémenté un script de réponse active personnalisé pour garantir un blocage réseau fiable et un déblocage automatique.

2.7.2 Partie Intégration de YARA avec Wazuh

- Objectifs :

- Détection d'un fichier malveillant
- Génération d'une alerte centralisée
- Suppression automatique du fichier

Pour cela, j'ai utilisé YARA pour la détection et Wazuh pour la supervision et la réponse.

Le principe de fonctionnement était le suivant :

1. YARA détecte un motif malveillant dans un fichier (détection par signature basée sur IOC)
2. L'événement est envoyé dans les logs système (syslog)
3. L'agent Wazuh collecte ce log
4. Le manager applique une règle personnalisée (règle personnalisée Wazuh ID 100100)
5. Une réponse automatique est déclenchée (Active Response)
6. Le fichier est supprimé

- Mise en place de la détection YARA

J'ai créé une règle YARA simple permettant de détecter une chaîne de caractères spécifique :

```
rule Test_Malware
{
  strings:
    $a = "malicious_test_string"

  condition:
    $a
}
```


Cette règle permet d'identifier un fichier contenant cette signature.

Lors du test :

```
echo "malicious_test_string" > /tmp/test.txt
```

YARA détectait correctement le fichier.

- Intégration avec Wazuh

Au lieu d'écrire dans un fichier spécifique, la sortie YARA a été envoyée vers syslog avec la commande :

```
sudo yara test_rule.yar /tmp/test.txt | logger -t yara
```

L'agent Wazuh surveille le fichier /var/log/syslog et transmet les événements au manager.

J'ai ensuite créé une règle personnalisée dans Wazuh pour détecter la signature Test_Malware et générer une alerte de niveau 10.

- Mise en place de l'Active Response

J'ai configuré une Active Response dans Wazuh afin que, lorsqu'une alerte correspondant à la règle 100100 est déclenchée, un script soit exécuté automatiquement sur l'agent.

Ce script devait :

- Lire l'alerte envoyée par Wazuh (au format JSON)
 - Extraire le chemin du fichier concerné
 - Supprimer le fichier
 - Enregistrer l'action dans les logs
- #### - Difficultés rencontrées
- Script initial bloqué car il attendait une entrée JSON
 - Nécessité de comprendre comment Wazuh transmet les données au script

La principale difficulté a été de comprendre que Wazuh n'envoie pas simplement un argument au script, mais un objet JSON complet contenant les informations de l'alerte.

Il a donc fallu analyser (parser) ce JSON afin d'en extraire le chemin du fichier.

Cette règle permet d'identifier un fichier contenant l'IOC malicious_test_string.

```
nad@infra-server:~$ cat /opt/yara-rules/test_rule.yar
rule Test_Malware
{
    strings:
        $a = "malicious_test_string"

    condition:
        $a
}
```

Règle personnalisée sur manager

```
nad@wazuh-server:~$ sudo cat /var/ossec/etc/rules/local_rules.xml
[sudo] Mot de passe de nad :
<group name="local,">

    <rule id="100100" level="10">
        <match>Test_Malware</match>
        <description>YARA detected Test_Malware on agent</description>
    </rule>

</group>
```

- Détection et suppression automatique

Création d'un fichier contenant une signature malveillante, détection par YARA puis envoi vers syslog.

La règle personnalisée Wazuh (ID 100100) déclenche une Active Response qui supprime automatiquement le fichier.

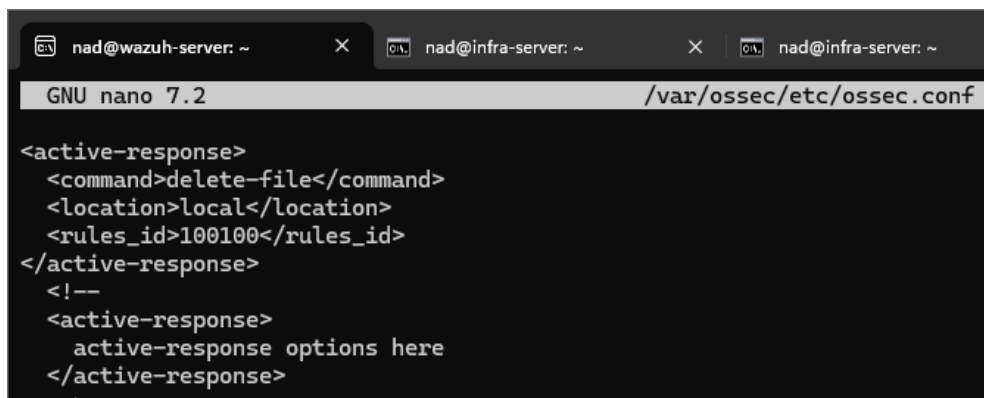
La commande ls confirme que le fichier n'existe plus.

```
nad@infra-server:~$ sudo systemctl restart wazuh-agent
nad@infra-server:~$ echo "malicious_test_string" > /tmp/test.txt
nad@infra-server:~$ sudo yara /opt/yara-rules/test_rule.yar /tmp/test.txt | logger -t yara
nad@infra-server:~$ ls /tmp/test.txt
ls: impossible d'accéder à '/tmp/test.txt': Aucun fichier ou dossier de ce nom
```

- Configuration de l'Active Response

Extrait du fichier ossec.conf montrant la configuration de l'Active Response associée à la règle 100100.

Cette configuration permet d'exécuter automatiquement la commande delete-file lorsqu'une détection YARA est confirmée.



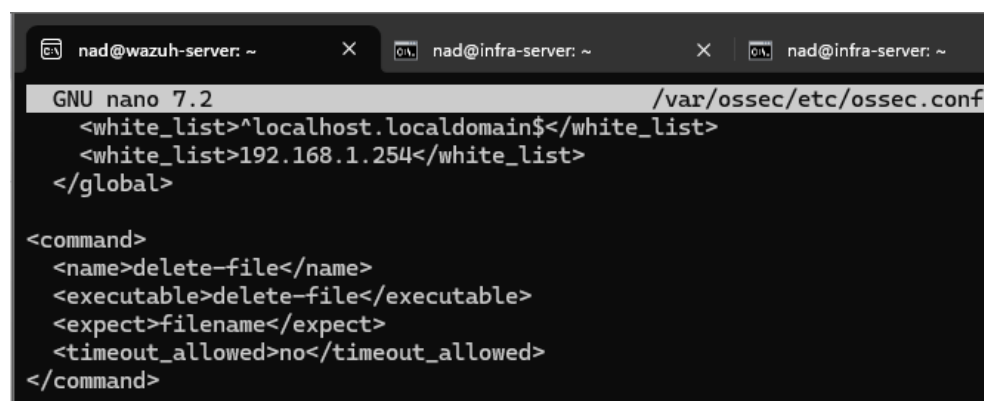
```
GNU nano 7.2 /var/ossec/etc/ossec.conf

<active-response>
  <command>delete-file</command>
  <location>local</location>
  <rules_id>100100</rules_id>
</active-response>
<!--
  <active-response>
    active-response options here
  </active-response>
```

- Déclaration de la commande personnalisée

Définition de la commande delete-file dans le fichier ossec.conf.

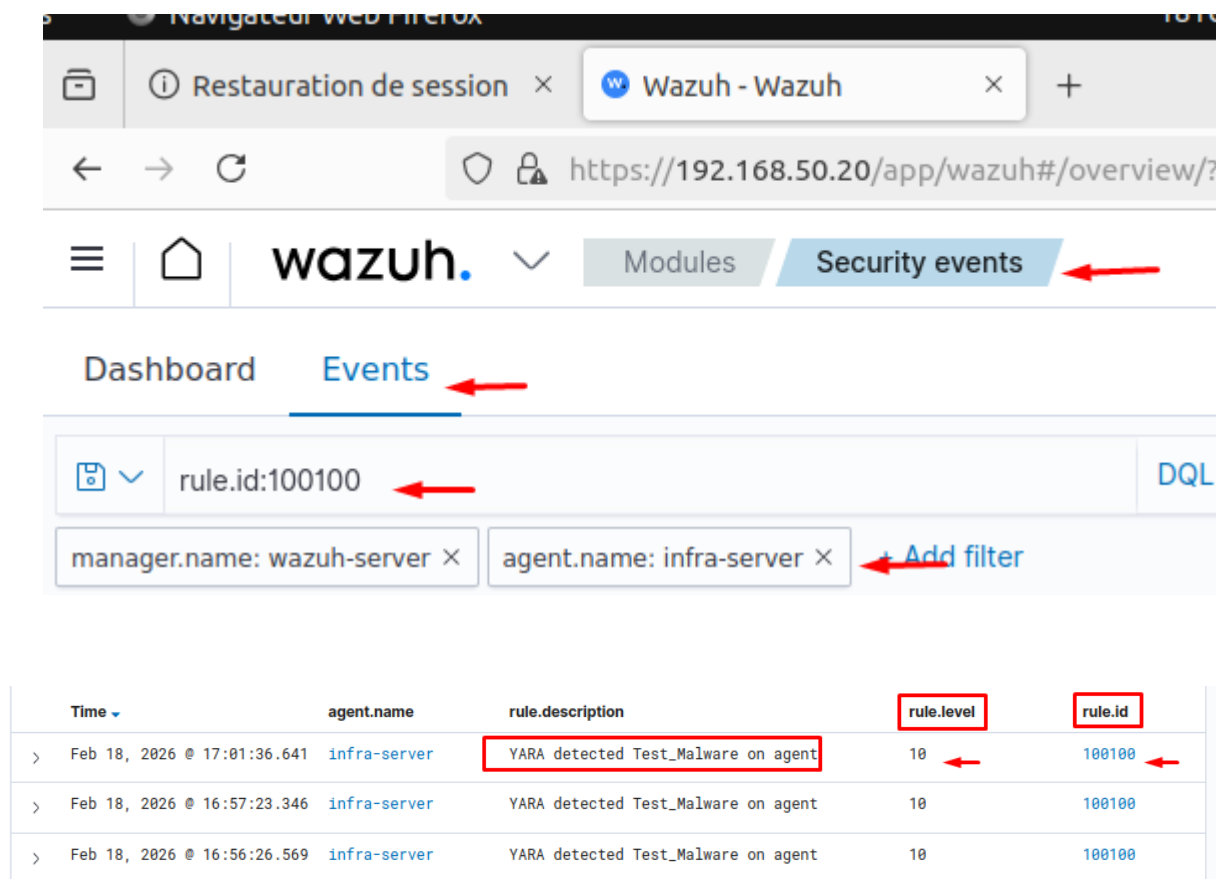
Cette commande exécute le script chargé de supprimer le fichier malveillant sur l'agent.



```
GNU nano 7.2 /var/ossec/etc/ossec.conf

  <white_list>^localhost.localdomain$</white_list>
  <white_list>192.168.1.254</white_list>
</global>

<command>
  <name>delete-file</name>
  <executable>delete-file</executable>
  <expect>filename</expect>
  <timeout_allowed>no</timeout_allowed>
</command>
```



Navigation: Restauration de session × Wazuh - Wazuh × +

Address: https://192.168.50.20/app/wazuh#/overview/?

Navigation: Modules Security events

Navigation: Dashboard Events

Filter: rule.id:100100 DQL

Filters: manager.name: wazuh-server × agent.name: infra-server × + Add filter

Time	agent.name	rule.description	rule.level	rule.id
> Feb 18, 2026 @ 17:01:36.641	infra-server	YARA detected Test_Malware on agent	10	100100
> Feb 18, 2026 @ 16:57:23.346	infra-server	YARA detected Test_Malware on agent	10	100100
> Feb 18, 2026 @ 16:56:26.569	infra-server	YARA detected Test_Malware on agent	10	100100